

Mobile and Embedded Computing

Lecture 2: compiled vs interpreted, null safety, Dart & Kotlin

Who's teaching this course

Dinu-Ștefan Rusu

dinu_stefan.rusu@upb.ro

Microsoft Teams course channel

rusudinu.com

45+

Flutter apps shipped

400k

installs across them

Appeared in, spoken at & partnered with



TODAY

What you should leave with



The compilation spectrum

AOT, JIT, bytecode VMs, interpreters, and why “compiled” describes an implementation, not a language



Sound null safety

How non-nullable-by-default removes a whole family of crashes, in Dart and, almost identically, in Kotlin



Dart you can actually write

Variables, functions, classes, collections and the null-aware operators: enough to read any Flutter code



What Flutter really is

One Dart codebase, one renderer, and why hot reload exists only in debug builds

HOW CODE RUNS

From source text to a running program

Compilers, interpreters, and the spectrum in between

Compiler and interpreter: what actually happens

A compiler translates your whole program ahead of time into instructions for one CPU and one OS. You ship the result; the translation is paid once, by you.

An interpreter is a program that reads your code and carries out its instructions step by step, every time it runs. You ship the source; the translation is paid on every run, on the user's device.

Almost nothing real is purely one or the other.

“Compiled” and “interpreted” are properties of a toolchain, not of a language. The same source can be run either way.

The same language, run differently

Python

CPython compiles to bytecode, then interprets it.
PyPy JIT-compiles the same source.

Java / Kotlin

Bytecode, interpreted at first, then JIT-compiled once a method gets hot.

JavaScript

V8 parses, interprets, then JIT-compiles, and de-optimizes when its guesses fail.

Dart

JIT while you develop. AOT for the build you ship.

The compilation spectrum

	AOT → native	JIT in a VM	Bytecode VM	Interpreter
Translated	Before you ship	While running	At load time	Line by line
You ship	Machine code	Source + VM	Bytecode + VM	Source + runtime
Start-up	Fastest	Needs warm-up	Medium	Starts instantly
Peak speed	High, fixed	Can beat AOT	Medium	Lowest
Examples	Go · Rust · Swift	JVM · V8 · ART	CPython	Bash · shell

Dart sits at two points on this spectrum at once: JIT while you develop, AOT for the build your users install.

Is compiled code faster?

There is a real gap, and it is entirely workload-dependent. A tight numeric loop shows the biggest difference; code dominated by I/O, syscalls or a native library shows almost none.

A JIT can beat an AOT compiler. It sees the actual types and hot branches at run time and inlines on that evidence, using information that does not exist ahead of time.

AOT wins where a phone needs it: cold start-up, memory footprint, and predictable latency.

In a Flutter frame the budget goes to layout, rasterization, I/O and garbage collection long before it goes to language dispatch.

Benchmark the workload you actually have. Numbers from any other benchmark describe someone else's program.

Not a fact

“Compiled languages are about 100× faster than interpreted ones.”

There is no single multiplier.

Ask instead: faster at what, measured how, on which device?

Why Dart compiles two different ways

Debug: JIT

```
flutter run
```

The Dart VM compiles your code as it runs

Hot reload: new code is swapped into the live isolate in under a second

Assertions on, DevTools attached, service protocol open

Slower, and a much larger binary

Release: AOT

```
flutter build apk --release
```

Compiled ahead of time to native ARM64 machine code

No VM and no interpreter ship inside your app

Fast cold start, smaller memory footprint, tree-shaken

No hot reload: there is nothing to reload into

Never judge performance from a debug build. Measure in profile or release mode: `flutter run --profile`.

Type errors and security are different things



Compile-time type errors

The compiler rejects `int x = 'hi'` before it ever ships. You get refactoring confidence and a smaller crash surface, at zero runtime cost.



Runtime failures

Nulls, index-out-of-range, a request that fails, malformed input. The type system narrows these, and null safety removes a whole family of them, but they cannot be eliminated.



Security vulnerabilities

Hard-coded API keys, a missing authorization check, injection, insecure local storage, over-broad permissions. A fully type-checked app can be completely insecure.

The one real overlap is memory safety. Dart, Kotlin, Go and Rust have no manual pointer arithmetic, so buffer overflows and use-after-free, the classic exploitable bugs of C and C++, cannot happen. That is memory safety, not type checking.

Security is a design and review discipline. We come back to it in Lecture 7, authentication and App Check.

NULL SAFETY

The billion-dollar mistake, and its fix

Why Dart and Kotlin make null part of the type

The billion-dollar mistake

```
// Java. This compiles cleanly, with no warning.  
String name = findUser(42);  
System.out.println(name.length());  
  
// findUser is allowed to return null.  
// NullPointerException at run time,  
// on a user's device, in production.
```

Where null came from

Tony Hoare put the null reference into ALGOL W in 1965 because it was easy to implement. In 2009 he called it his “billion-dollar mistake”.

Non-nullable by default, since:

Swift: 2014, optionals

Kotlin: 2016, from 1.0

Dart: 2021, from 2.12

TypeScript: strictNullChecks

C#: nullable reference types

Null is not a value problem, it is a type problem. If the type cannot say “maybe absent”, the compiler cannot help you.

Dart null safety: the operators

```
// Non-nullable by default: String name = null; is an error.
String name = 'Ada';

// Add ? and the type is allowed to hold null.
String? nickname;

// ?. calls only if non-null; ?? supplies a fallback.
final shown = nickname?.trim() ?? name;

// ??= assigns only when the target is null.
nickname ??= 'anon';

// ! asserts non-null; it throws if you were wrong.
final n = nickname!.length;
```

Four symbols, and that is most of it

? the type may be null

?. guarded access

?? fallback value

??= fallback assignment

! assertion, not a fix

Every ! is a claim the compiler cannot check for you. If a file is full of them, the types are wrong; fix the types instead.

What “sound” means, and the escape hatches

```
// Flow analysis promotes a nullable to non-nullable.
String greet(String? name) {
  if (name == null) return 'Hello, stranger';
  return 'Hello, ${name.toUpperCase()}'; // no ! needed
}

// late: I promise to assign this before anything reads it.
class Repo {
  late final Database db;
  Repo() { db = openDatabase(); }
}

// A broken promise throws LateInitializationError.
```

Sound = the guarantee holds

A non-nullable Dart variable can never hold null at run time. This is a guarantee, not a strong tendency.

So the compiler stops emitting null checks for it. Sound null safety makes release builds smaller and faster, not only safer.

Use late for dependency injection and expensive lazy fields, not to silence the analyzer.

Null safety in Dart and Kotlin, side by side

Dart

```
String? findEmail(int id) { ... }

void notify(int id) {
  final email = findEmail(id);
  if (email == null) return;
  send(email);
  // promoted to String
}
```

Kotlin

```
fun findEmail(id: Int): String? { ... }

fun notify(id: Int) {
  val email = findEmail(id)
  if (email == null) return
  send(email)
  // smart cast to String
}
```

?.
??
??=
!
late final

?.
?:
(none)
!!
lateinit var

Learn it once. The concept, and most of the syntax, transfers straight to Kotlin, and to Swift and Rust after that. One real difference: Kotlin's `lateinit` works on `var` only, while Dart allows `late final`.

DART & KOTLIN

The language you will write this semester

Dart 3.12, with Kotlin 2.4 alongside it for comparison

Variables: var, final, const

```
// var infers its type from the initializer.  
var count = 0;  
String title = 'Lecture 2';  
  
// final: assigned once, at run time.  
final now = DateTime.now();  
  
// const: assigned once, by the compiler, baked into  
// the binary. Everything in it must be known statically.  
const frameBudgetMs = 1000 / 120;  
  
// String interpolation, not concatenation.  
print('$title: $frameBudgetMs ms per frame');
```

Read it this way

var

still statically typed: the type is inferred, not absent.

final

the binding cannot be reassigned. The object it points at may still change.

const

a compile-time constant. In Flutter, const widgets are built once and reused, which is a real performance lever.

dynamic

opts out of static checking entirely. You will almost never want it.

Functions and parameters

```
// Positional parameters; => is a one-expression body.
int add(int a, int b) => a + b;

// Optional positional parameters, with a default.
String tag(String n, [String suffix = '']) => '$n$suffix';

// Named parameters: this is the Flutter style.
String card({
  required String title,
  String? subtitle,
  int maxLines = 2,
}) => title;

card(title: 'Dart', maxLines: 3);
```

Why named parameters matter

Every Flutter widget constructor uses them. Call sites read as prose instead of as a row of anonymous positional arguments.

required is a compile-time obligation, not a runtime check.

Functions are first-class values: pass them, store them, return them. That is what a callback like `onPressed` actually is.

Classes, constructors and getters

```
class Lecture {  
  final int number;  
  final String title;  
  final String? room;  
  
  // this.x in the parameter list assigns the field.  
  Lecture({required this.number, required this.title,  
          this.room});  
  
  // A getter: computed, called without parentheses.  
  String get label => 'L$number: $title';  
}  
  
final l = Lecture(number: 2, title: 'Languages');
```

Also worth knowing

Named constructors:

```
Lecture.lab(this.number)  
          : title = 'Lab', room = null;
```

Every class gets a default toString, ==, and hashCode, and == is identity by default.

Two Lecture objects with the same fields are not equal. Lecture 4 fixes that with Equatable.

Collections and control flow

```
final topics = <String>['Dart', 'Kotlin', 'Flutter'];
final seen = <String>{};
final points = <String, int>{'exam': 3, 'lab': 3};

for (final t in topics) { seen.add(t); }

// Collection-if and collection-for build lists inline.
final menu = [
  'Home',
  if (isAdmin) 'Admin',
  for (final t in topics) 'Lab: $t',
];

topics.where((t) => t.length > 4).forEach(print);
```

You will see this everywhere

List, Set and Map are the three built-ins. The angle brackets are the element types.

Collection-if and collection-for are how Flutter builds a list of children conditionally, without a helper function and without a temporary list.

where, map, fold and forEach are lazy Iterable operations; call `toList()` when you need a List back.

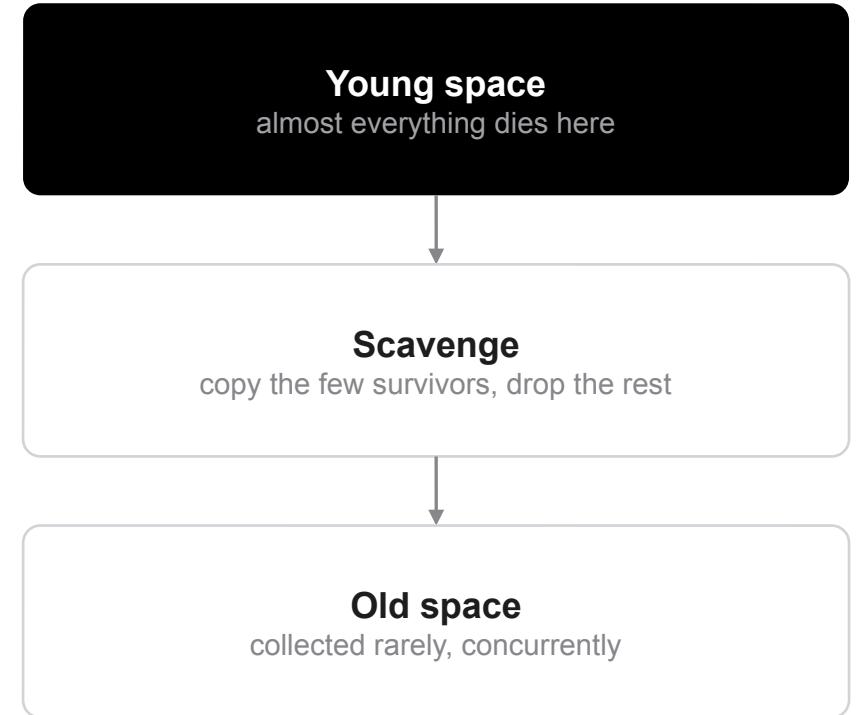
Garbage collection, in one slide

Dart is garbage collected: you never free memory. The VM reclaims objects that nothing can reach any more.

The generational hypothesis: most objects die young. Flutter proves it: a rebuild allocates thousands of short-lived Widget objects and discards them a frame later.

So the heap is split. The young space is collected constantly and cheaply. Survivors are promoted to the old space, which is collected rarely and concurrently with your app.

Why it matters on a phone: a pause longer than one frame budget, 8.3 ms at 120 Hz, is a visible dropped frame, and every collection costs CPU cycles and therefore battery. Allocate less inside build(), use const constructors, and keep heavy work off the UI isolate (→ Lecture 3).



The bookkeeping that keeps this correct, remembered sets and write barriers, is beyond the scope of week 2.

Kotlin and Dart: the same shapes

Dart

```
class User {  
    final String name;  
    final int age;  
    const User(this.name, this.age);  
}  
  
final u = User('Ada', 36);  
var count = 0;
```

Kotlin's data class gives you `==`, `hashCode`, `copy` and `toString` for free. In Dart you write them by hand, or generate them with `Equatable` or `freezed` (→ Lecture 4).

Kotlin

```
data class User(  
    val name: String,  
    val age: Int,  
)  
  
val u = User("Ada", 36)  
var count = 0
```

The same spectrum, again: Kotlin/JVM ships bytecode that Android's ART compiles; Kotlin/Native compiles ahead of time for iOS. Shared modules: Lecture 11.

FLUTTER

What Flutter actually is

One Dart codebase, one renderer, and a build mode that changes how the code runs

What Flutter is, and what it draws

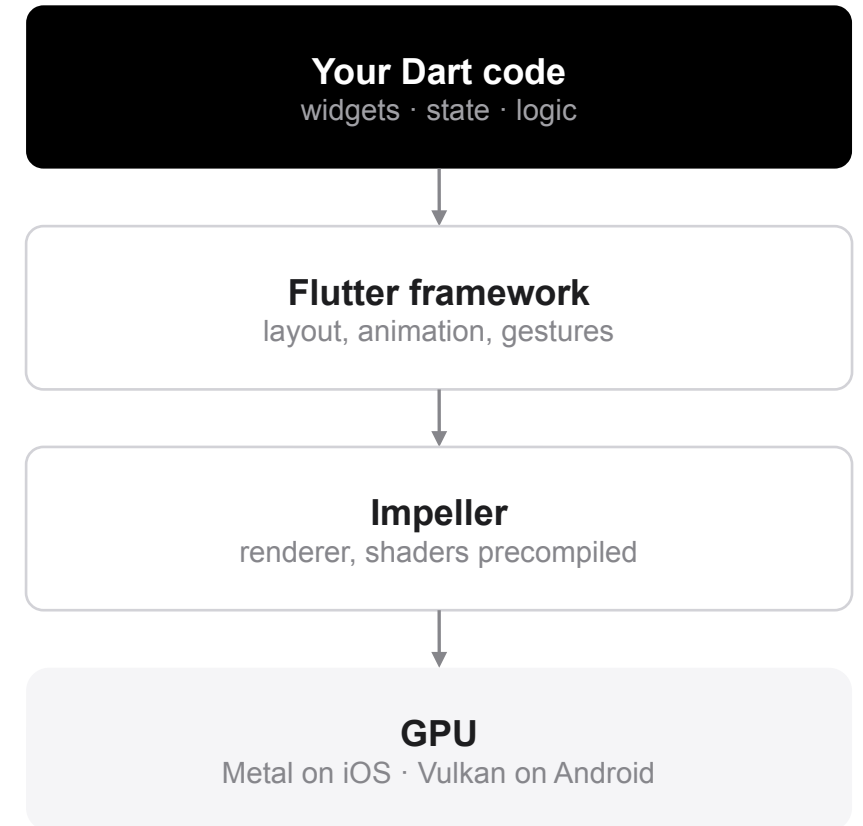
Google's open-source UI toolkit: one Dart codebase for Android, iOS, web and desktop.

Flutter does not use the platform's own widgets. It ships its own widget library and draws every pixel itself, through the Impeller renderer.

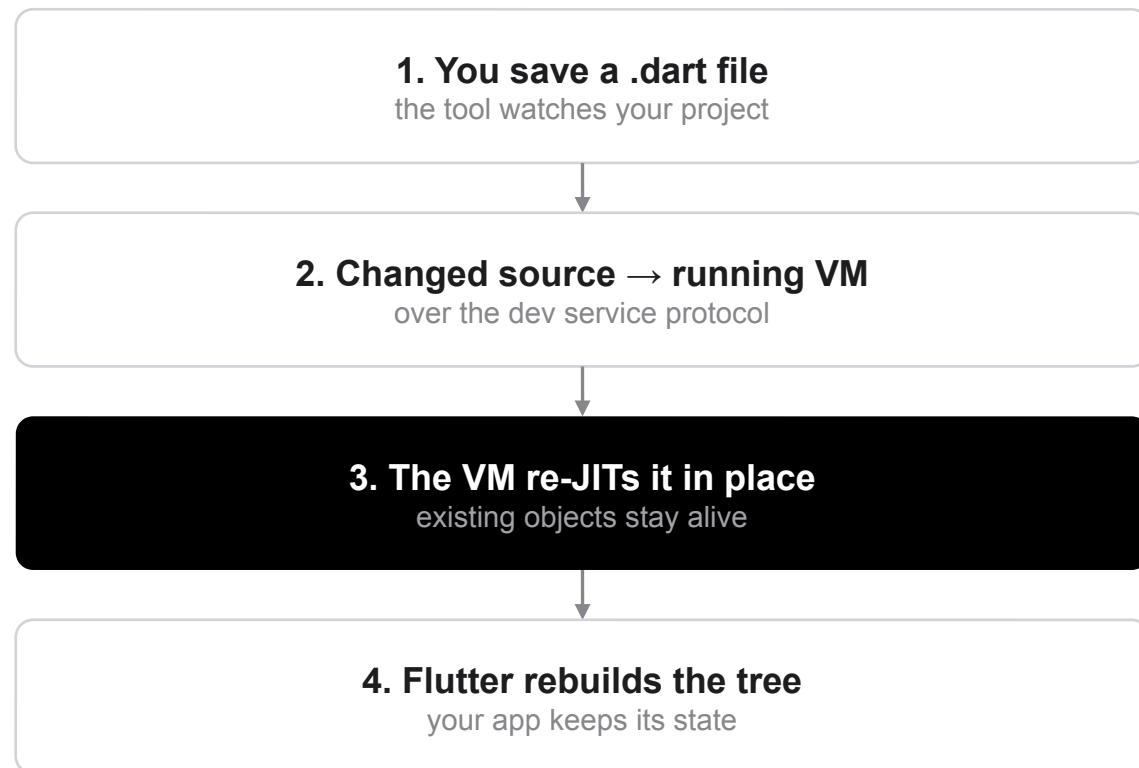
That is why a Flutter app looks identical on both platforms, and why the platform's native controls are not automatically available.

It reaches the OS (camera, sensors, BLE, files) through platform channels (→ Lecture 12).

A release build is Dart, AOT-compiled to native ARM code. There is no interpreter and no JavaScript inside your app.



Hot reload, and why release builds cannot



What hot reload cannot do

Change `main()` or anything that only runs at start-up

Change an enum into a class, or a generic type declaration

Fix a compile error: the build has to succeed first

Work in a release build: it is AOT machine code, and there is no VM to load new code into.

`r = reload` · `R = restart (state lost)`

Hot reload is the payoff of the JIT half of the spectrum. It is a development-time feature, and it is the reason Dart kept its VM.

Your toolchain: install, create, run

```
# docs.flutter.dev/install, then Android Studio for the  
# Android SDK and an emulator, Xcode on macOS for iOS,  
# and VS Code or IntelliJ with the Dart + Flutter plugins.
```

```
flutter doctor
```

```
# checks every dependency and names what is missing
```

```
flutter create my_app && cd my_app
```

```
flutter run
```

```
# debug build: JIT, hot reload (r), hot restart (R)
```

```
flutter run --release
```

```
# AOT machine code: what your users actually run
```

In the lab this week

Lab 2 walks the whole install end to end, on Windows, macOS and Linux.

Run flutter doctor before you ask for help; it names the missing piece in one line.

An Android emulator is enough for every lab. A real device is faster, and the only way to test sensors.

iOS builds need macOS and Xcode; the labs never require them.

Before next week



Recap

Compilation is a spectrum: Dart sits at two points on it

Types catch type errors; security is a separate discipline

Non-nullable by default: ? ?? ! late, and Kotlin maps 1:1

Flutter = your Dart code, drawn by Impeller, AOT in release



This week

Lab 2: install the SDK, run flutter doctor, create and run an app

Rewrite the null-unsafe snippet from the handout using ?, ?? and promotion

Push the lab on a branch and open a PR, the Lecture 1 loop

Form your project team (max 3) and claim a concept-presentation topic



Read more

dart.dev/language

dart.dev/null-safety

dart.dev/effective-dart

kotlinlang.org/docs/null-safety.html

docs.flutter.dev/get-started